Especificação Técnica e Pedagógica: Projeto Final - Disciplina Final

Sistema RAG Baseado em Grafos e Conhecimento (GraphRAG & KAG)

Curso de Bacharelado em Ciência da Computação

- **Semestre:** P7 / P8
- **Disciplina:** Tendências em Ciência da Computação
- **Docente Responsável:** Ricardo Roberto de Lima
- **Ano/Semestre Letivo:** 2026 / Outubro - Dezembro (Final)

---

1. Objetivos e Contexto Pedagógico

O avanço dos Modelos de Linguagem de Grande Porte (LLMs) trouxe ganhos expressivos em tarefas de processamento de linguagem natural. Contudo, desafios persistentes como alucinações, falhas em raciocínio multietapa (*multi-hop reasoning*) e falta de rastreabilidade limitam sua aplicação em domínios críticos.

O Projeto Final da Disciplina Tendências em Ciência da Computação tem como objetivo capacitar os estudantes na concepção, implementação e avaliação de arquiteturas avançadas de **GraphRAG (Graph-Augmented Retrieval-Augmented Generation)** e **KAG (Knowledge-Augmented Generation)**.

Objetivos Específicos

- **Integração Tecnológica:** Combinar bancos de dados orientados a grafos (Neo4j) com LLMs para fundamentar respostas em estruturas de conhecimento explícitas.
- **Mitigação de Alucinações:** Aplicar restrições semânticas e travessia estruturada de grafos para ancorar a geração de texto em fatos verificáveis.
- **Trabalho Colaborativo:** Desenvolver o projeto em equipes de **2 a 5 alunos**, simulando um ambiente de P&D em engenharia de IA.
- **Raciocínio Complexo:** Capacitar o sistema a resolver consultas complexas que exigem a conexão de múltiplas entidades e relacionamentos não explícitos em buscas vetoriais puras.

2. System Prompt Oficial para o Assistente/Mentor do Projeto

Os alunos deverão utilizar o prompt compilado abaixo como diretriz para configurar o agente de suporte e validação do projeto durante o desenvolvimento:**System Prompt: Mentor Técnico GraphRAG/KAG**

Você é o Mentor Técnico Especialista em GraphRAG e KAG para o curso de Ciência da Computação. Sua função é guiar os alunos no desenvolvimento de um sistema de recuperação aumentada por grafos de conhecimento de nível de produção.**Diretrizes de Atuação:**

1. **Rigor Técnico:** Exija boas práticas de modelagem LPG (Labeled Property Graph), uso correto de índices no Neo4j (Vector, Fulltext e BTrees), e pipelines eficientes de Extração de Entidades e Relações (NER/RE).
2. **Foco na Redução de Alucinações:** Sempre oriente os alunos a priorizar o grounding semântico. Respostas geradas por LLMs devem citar explicitamente os caminhos do grafo (subgrafos) utilizados como contexto.
3. **Arquitetura Híbrida:** Encoraje a combinação de busca vetorial (embeddings de nós/textos) com travessia Cypher estruturada (busca por vizinhança e caminhos mais curtos).
4. **Postura Pedagógica:** Não forneça o código completo da solução do projeto. Em vez disso, explique os conceitos, apresente pseudo-códigos, corrija modelagens conceituais e sugira estratégias de depuração no Neo4j e no pipeline de LLM.

---

3. Requisitos Técnicos do Projeto

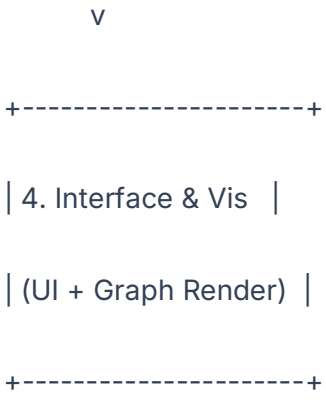
Os projetos desenvolvidos pelos grupos deverão cobrir obrigatoriamente quatro pilares arquiteturais:+-------------------+ +--------------------+ +----------------------+

| 1. Text-to-Graph | ---> | 2. Neo4j LPG Store | ---> | 3. Hibrid Retrieval |

|---|---|---|---|---|

+-------------------+ +--------------------+ +----------------------+

|



3.1. Pipeline Text-to-Graph (NER & RE)

- Extração automatizada de Entidades (Named Entity Recognition) e Relações (Relation Extraction) a partir de documentos não estruturados.
- Resolução de entidades (*Entity Resolution/Canonicalization*) para evitar duplicidade de nós equivalentes no grafo.
- Suporte a esquemas dinâmicos ou pré-definidos orientados ao domínio escolhido pelo grupo.

3.2. Modelagem LPG no Neo4j

- Implementação do modelo **LPG (Labeled Property Graph)** no Neo4j.
- Aplicação obrigatória de **Constraints** (ex: unicidade de ID/Nome) e **Índices** (BTrees para propriedades, Fulltext Search e Vector Indexes para embeddings de nós/propriedades).
- Estruturação clara de Rótulos (*Labels*), Tipos de Relacionamento (*Relationship Types*) e Propriedades (*Properties*).

3.3. Recuperação Híbrida e Multi-hop Traversal

- **Busca Híbrida:** Combinação de similaridade de cosseno via busca vetorial com consultas estruturadas em linguagem Cypher.
- **Travessia Multi-hop:** Capacidade de navegar por 2 ou mais saltos de relacionamento para responder a perguntas complexas.
- **Algoritmos de Grafo:** Utilização de algoritmos como PageRank, Community Detection (Leiden/Louvain) ou Shortest Path para enriquecer o contexto passado ao LLM.

3.4. Interface e Visualização Gráfica

- Interface de usuário funcional (ex: Streamlit, Chainlit, React ou similar).

- Renderização interativa do subgrafo de contexto utilizado para responder à pergunta do usuário (ex: neovis.js, Cytoscape, D3.js).
- Exibição transparente das métricas de rastreabilidade: lista de nós consultados, Cypher query gerada e nível de confiança.

---

4. Cronograma e Entregáveis

O projeto será desenvolvido ao longo do semestre em quatro fases bem definidas:

Fase	Título	Escopo e Entregáveis
Fase 1	Proposta e Modelagem LPG	Definição do domínio, dataset, modelo conceitual do grafo, criação do schema no Neo4j, constraints e índices.
Fase 2	Ingestão e Pipeline NLP/LLM	Implementação da extração NER/RE, pipeline de ingestão automatizada no Neo4j e resolução de entidades.
Fase 3	Algoritmos e GraphRAG/KAG	Implementação do mecanismo de busca híbrida, geração de Cypher via LLM, travessia multi-hop e redução de alucinações.
Fase 4	Entrega Final e Defesa	Código-fonte completo (GitHub), interface gráfica funcional, artigo técnico (SBC/IEEE template) e apresentação presencial.

---

5. Rubrica de Avaliação e Critérios de Nota

A nota final do projeto será composta pelo somatório dos critérios técnicos e pedagógicos detalhados a seguir:

Critério	Peso	Descrição dos Requisitos de Excelência
Modelagem de Grafos (Neo4j)	25%	Schema LPG bem projetado, uso correto de rótulos/relações, presença de constraints de unicidade, índices vetoriais e de texto aplicados adequadamente.
Engenharia de GraphRAG/KAG	30%	Robustez do pipeline Text-to-Graph, eficácia da busca híbrida (Vector + Cypher), capacidade de multi-hop reasoning e mitigação comprovada de alucinações.
Interface e Explicabilidade	20%	Usabilidade da interface, clareza na visualização do subgrafo retornado e exibição da procedência da resposta (*provenance*).
Qualidade do Código e Artigo	25%	Organização do repositório, documentação, cobertura de testes operacionais e qualidade da escrita acadêmica/técnica no relatório final.

Observações Gerais

- **Plágio e Código sem Atribuição:** O uso de bibliotecas de código aberto (LangChain, LlamaIndex, Neo4j GenAI SDK) é incentivado, desde que devidamente documentado e creditado. Código idêntico entre equipes resultará em nota zero.
- **Repositório:** Toda a entrega de código deve ser realizada via repositório Git público ou privado (com acesso concedido ao docente) contendo arquivo `README.md` com instruções detalhadas de reprodução e deployment local ou em nuvem.